

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ
им. А.Н. ТИХОНОВА НИУ ВШЭ**

Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

**ИНТЕРАКТИВНОЕ ПРИЛОЖЕНИЕ
ДЛЯ ИЗУЧЕНИЯ АНГЛИЙСКОГО ЯЗЫКА**

«Language Learning App»

Мобильное приложение на базе Flutter

ОТЧЁТ ПО ПРОЕКТУ

РАЗРАБОТЧИКИ:

Амана Бикиливе Паскаль
Старченко Денис
Абашилов Магомед

Группа:

БИВ 255

РУКОВОДИТЕЛЬ:

Тихонов Р.А.

Москва, 2026 г.

СОДЕРЖАНИЕ

1	Введение	3
1.1	Актуальность темы	3
1.2	Цель проекта	3
1.3	Задачи проекта	3
1.4	Структура работы	4
2	Обзор существующих решений	5
2.1	Анализ рынка приложений для изучения языков	5
2.2	Duolingo	5
2.3	Babbel	5
2.4	Lingualeo	6
2.5	Сравнительный анализ	6
2.6	Выводы по обзору	6
3	Теоретические основы	8
3.1	Фреймворк Flutter	8
3.2	Язык программирования Dart	8
3.3	Управление состоянием	8
3.4	SharedPreferences	9
3.5	HTTP-запросы и REST API	9
3.6	Web Speech API и межплатформенное взаимодействие	9
3.7	Принципы геймификации в образовании	9
4	Архитектура и реализация	10
4.1	Общая архитектура приложения	10
4.2	Структура проекта	10
4.3	База данных слов	11
4.4	Реализация игровых режимов	12
4.4.1	Режим «Сопоставление пар» (Matching)	12
4.4.2	Режим «Викторина» (Quiz)	12
4.4.3	Режим «Перетаскивание» (Drag & Drop)	12
4.5	Система прогресса и звёзд	13
4.6	Тёмная и светлая тема	13
4.7	Голосовое распознавание	14
4.8	Серверная часть	14
4.9	Аудиосистема	14

5	Тестирование и результаты	15
5.1	Методология тестирования	15
5.2	Результаты функционального тестирования	15
5.3	Тестирование пользовательского интерфейса	15
5.4	Тестирование совместимости	16
5.5	Выявленные проблемы и их решение	16
5.6	Производительность	16
6	Развертывание и использование	17
6.1	Требования к окружению	17
6.2	Клонирование репозитория	17
6.3	Установка зависимостей Flutter	17
6.4	Запуск приложения в веб-режиме	17
6.5	Запуск серверной части	18
6.6	Использование приложения	18
6.7	Возможные неполадки и их устранение	18
7	Заключение	19
7.1	Итоги работы	19
7.2	Практическая значимость	19
7.3	Возможные улучшения	19
7.4	Заключительные выводы	20
	Список использованных источников	21

1 Введение

1.1 Актуальность темы

В эпоху глобализации английский язык стал универсальным средством международной коммуникации. По данным Ethnologue (2025), английским языком владеют более 1,5 миллиарда человек по всему миру, из которых около 400 миллионов являются носителями, а остальные используют его как второй язык [1]. Для русскоговорящих студентов и специалистов владение английским открывает доступ к международным образовательным программам, научным публикациям и карьерным возможностям.

Традиционные методы изучения языка, такие как аудиторные занятия и работа с учебниками, имеют ряд ограничений: фиксированное расписание, недостаток индивидуального подхода, ограниченные возможности для практики произношения и отсутствие немедленной обратной связи. Современные технологии, в частности мобильные приложения и методы искусственного интеллекта, позволяют преодолеть эти ограничения.

Данный проект направлен на разработку кроссплатформенного интерактивного приложения для изучения английского языка, ориентированного на русскоговорящих пользователей. Приложение сочетает в себе элементы геймификации, систему отслеживания прогресса и разнообразные типы упражнений для эффективного запоминания лексики.

Исходный код проекта размещён в публичном репозитории GitHub: <https://github.com/EfecTr0/Duolingo> (директория `game_app`).

1.2 Цель проекта

Целью данного проекта является разработка кроссплатформенного мобильного приложения на базе фреймворка Flutter, обеспечивающего интерактивное и геймифицированное обучение английскому языку для русскоговорящих пользователей.

1.3 Задачи проекта

Для достижения поставленной цели были сформулированы следующие задачи:

1. Провести анализ существующих решений в области мобильных приложений для изучения языков
2. Изучить теоретические основы разработки на Flutter и Dart
3. Спроектировать архитектуру приложения
4. Реализовать пользовательский интерфейс на русском языке
5. Разработать базу данных слов с четырьмя уровнями сложности
6. Реализовать три типа интерактивных упражнений
7. Внедрить систему сохранения прогресса
8. Провести тестирование приложения на различных платформах

1.4 Структура работы

Настоящий отчёт состоит из введения, пяти глав, заключения и списка использованных источников. В первой главе проводится обзор существующих решений и анализ аналогов. Вторая глава посвящена теоретическим основам используемых технологий. Третья глава описывает архитектуру и реализацию приложения. Четвёртая глава содержит описание процесса тестирования и полученные результаты. Пятая глава содержит инструкции по развертыванию и использованию приложения.

2 Обзор существующих решений

2.1 Анализ рынка приложений для изучения языков

Рынок образовательных мобильных приложений демонстрирует устойчивый рост. По данным Statista (2025), объём мирового рынка приложений для изучения языков в 2025 году составил \$8,2 миллиарда и прогнозируется на уровне \$15,3 миллиарда к 2030 году [2]. Наиболее популярными приложениями в данном сегменте являются Duolingo, Babbel, Busuu, Lingualeo и Memrise.

2.2 Duolingo

Duolingo является самым популярным в мире приложением для изучения языков с более чем 500 миллионами загрузок. Приложение использует модель freemium и предлагает изучение более 40 языков через короткие интерактивные уроки [3].

Преимущества:

- Высокий уровень геймификации (очки опыта, уровни, достижения)
- Адаптивная система повторения (spaced repetition)
- Поддержка большого количества языковых пар
- Наличие веб-версии и мобильных приложений

Недостатки:

- Ограниченные возможности для практики произношения
- Отсутствие режима соревнования с друзьями в реальном времени
- Упрощённая система упражнений (преимущественно перевод и выбор слов)
- Отсутствие глубокой персонализации на основе анализа ошибок

2.3 Babbel

Babbel — платное приложение, ориентированное на взрослую аудиторию. Предлагает курсы, разработанные профессиональными лингвистами, с акцентом на практическое применение языка [4].

Преимущества:

- Качественный контент, созданный лингвистами
- Практическая направленность уроков
- Встроенная система распознавания речи

Недостатки:

- Высокая стоимость подписки
- Отсутствие бесплатного функционала
- Нет режима PvP (игрок против игрока)
- Меньшее разнообразие игровых механик

2.4 Lingualeo

Lingualeo — российское приложение для изучения английского языка, популярное среди русскоговорящей аудитории. Предлагает обучение через чтение, аудирование и грамматические упражнения.

Преимущества:

- Интерфейс на русском языке
- Большая база контента (тексты, видео, аудио)
- Адаптировано под русскоговорящих пользователей

Недостатки:

- Устаревший дизайн интерфейса
- Ограниченная кроссплатформенность
- Отсутствие режима реального времени с друзьями
- Слабая интеграция с современными AI-технологиями

2.5 Сравнительный анализ

Критерий	Duolingo	Babbel	Lingualeo	Наше приложение
Интерфейс на русском	Нет	Нет	Да	Да
Геймификация	Высокая	Средняя	Средняя	Высокая
Распознавание речи	Базовое	Да	Базовое	Да
Режим PvP	Нет	Нет	Нет	Да
Кроссплатформенность	Да	Да	Частично	Да
Тёмная тема	Нет	Нет	Нет	Да
Сохранение прогресса	Да	Да	Да	Да
Открытый исходный код	Нет	Нет	Нет	Да

Таблица 1: Сравнительный анализ приложений для изучения языков

2.6 Выводы по обзору

Проведённый анализ показывает, что несмотря на наличие большого количества приложений для изучения языков, существует ниша для продукта, который объединяет:

- Полностью русскоязычный интерфейс

- Разнообразные игровые механики
- Режим соревнования с друзьями (PvP)
- Голосовое распознавание речи
- Открытый исходный код

Разрабатываемое приложение призвано заполнить этот пробел, предлагая русскоговорящим пользователям интерактивный и увлекательный способ изучения английской лексики.

3 Теоретические основы

3.1 Фреймворк Flutter

Flutter — это кроссплатформенный UI-фреймворк с открытым исходным кодом, разработанный компанией Google. Выпущенный в 2018 году, Flutter позволяет создавать нативные приложения для мобильных (Android, iOS), веб-платформ и десктопов (Windows, macOS, Linux) из единой кодовой базы [5].

Ключевые особенности Flutter:

- **Язык Dart** — объектно-ориентированный язык программирования, разработанный Google, оптимизированный для построения пользовательских интерфейсов
- **Декларативный UI** — интерфейс описывается как функция состояния, что упрощает разработку и отладку
- **Виджеты** — все элементы интерфейса являются виджетами, организованными в древовидную структуру
- **Hot Reload** — возможность мгновенного обновления интерфейса без перезапуска приложения
- **Собственный движок рендеринга** — Flutter использует Skia для отрисовки интерфейса, что обеспечивает высокую производительность

3.2 Язык программирования Dart

Dart — клиент-ориентированный язык программирования, оптимизированный для быстрой разработки приложений на всех платформах. Версия Dart 3.x, использованная в проекте, включает улучшенную систему типов, records, pattern matching и другие современные возможности [6].

Преимущества Dart:

- Статическая типизация с поддержкой type inference
- Асинхронное программирование через async/await
- Компиляция как в нативный код, так и в JavaScript (для веба)
- Богатая стандартная библиотека

3.3 Управление состоянием

В приложении используется встроенный механизм управления состоянием Flutter — `setState()` в сочетании с `StatefulWidget`. Для каждой игровой механики создаётся отдельный экран-виджет, который хранит своё состояние (выбранные слова, очки, прогресс) и перестраивает интерфейс при его изменении.

3.4 SharedPreferences

SharedPreferences — это плагин Flutter, предоставляющий простое хранилище типа «ключ-значение» для сохранения данных на устройстве [7]. В проекте SharedPreferences используется для сохранения прогресса пользователя (количество заработанных звёзд) между сессиями.

API SharedPreferences:

- `getInt (key)` — получение целочисленного значения по ключу
- `setInt (key, value)` — сохранение целочисленного значения
- `remove (key)` — удаление значения по ключу

3.5 HTTP-запросы и REST API

Для взаимодействия с серверной частью (Python backend) используется пакет `http` [8], который предоставляет удобный интерфейс для выполнения HTTP-запросов. Приложение отправляет GET и POST запросы для аутентификации пользователей, получения списка друзей и синхронизации игровых данных в режиме PvP.

3.6 Web Speech API и межплатформенное взаимодействие

Для реализации голосового ввода в веб-версии приложения используется Web Speech API — встроенный API браузера для распознавания речи [9]. Взаимодействие между Dart и JavaScript осуществляется через пакет `dart:js`, который позволяет вызывать JavaScript-функции из Dart-кода и наоборот.

3.7 Принципы геймификации в образовании

Геймификация — применение игровых элементов и техник в неигровых контекстах, в частности, в образовании. Исследования (Deterding et al., 2011 [12]; Namari et al., 2014 [13]) показывают, что геймификация повышает вовлечённость и мотивацию учащихся. В разработанном приложении используются следующие элементы геймификации:

- **Система звёзд** — награда за успешное выполнение упражнений
- **Уровни сложности** — постепенное усложнение материала
- **Мгновенная обратная связь** — визуальное подтверждение правильных/неправильных ответов
- **Соревновательный режим** — PvP-батлы с друзьями

4 Архитектура и реализация

4.1 Общая архитектура приложения

Приложение построено по модульному принципу. Каждый экран и игровой режим реализован в виде отдельного виджета Flutter, что обеспечивает изоляцию логики и лёгкость поддержки кода.

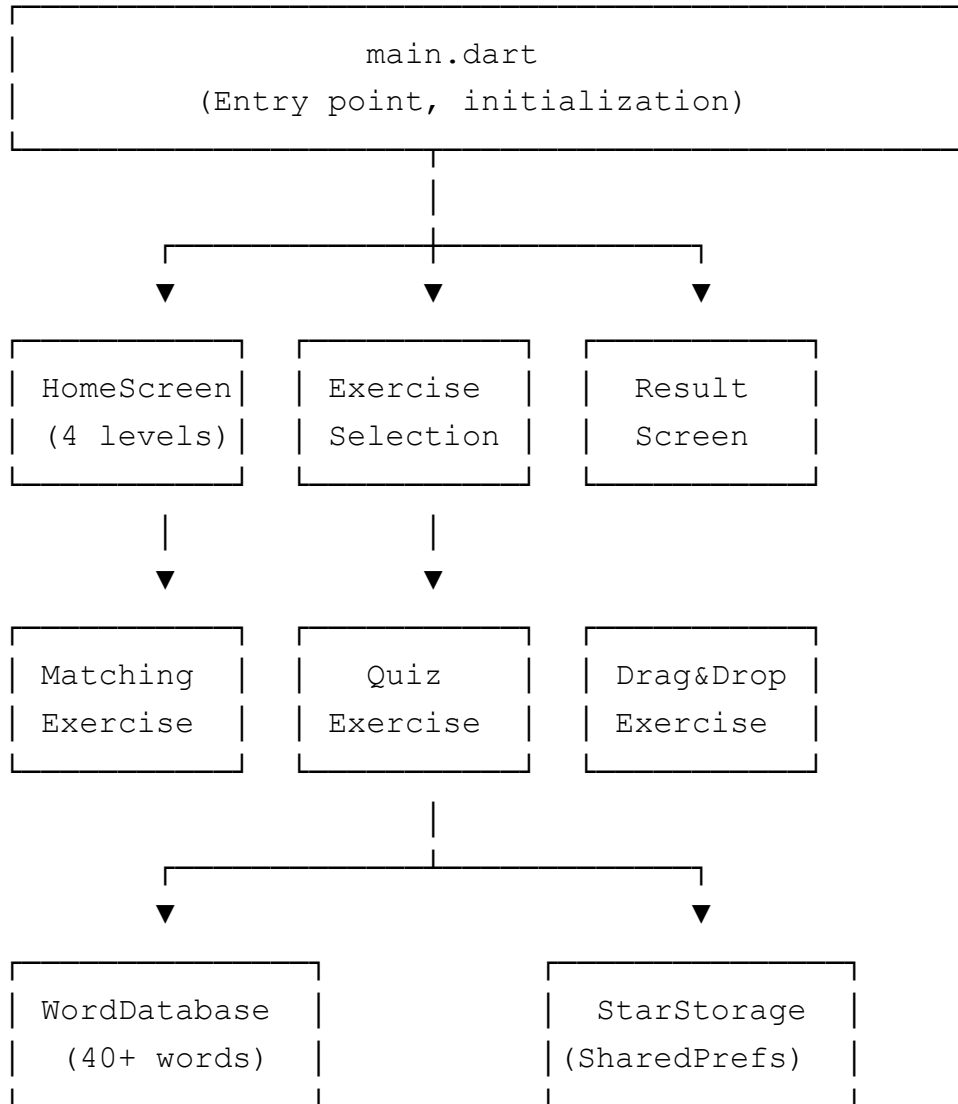


Рис. 1: Архитектура приложения

4.2 Структура проекта

Проект организован в соответствии с лучшими практиками Flutter-разработки. Исходный код размещён в директории `lib/` и состоит из следующих файлов:

Файл	Назначение
main.dart	Точка входа в приложение, конфигурация темы, JavaScript-интероп
login_screen.dart	Экран аутентификации пользователя
main_screen.dart	Главный экран с навигацией по разделам
api_service.dart	Сервис для взаимодействия с серверным API
data/player.dart	Модель данных игрока
data/words.dart	Модель данных слов и база слов
screens/menu_screen.dart	Меню выбора режима тренировки
screens/game_screen.dart	Базовый игровой экран
screens/matching_cards_game.dart	Режим сопоставления карточек
screens/match_game_screen.dart	Экран PvP-матча
screens/mixed_game_screen.dart	Смешанный режим упражнений
screens/pvp_game_screen.dart	PvP-режим игры с друзьями
screens/profile_screen.dart	Экран профиля пользователя
screens/friends_screen.dart	Экран управления друзьями
screens/settings_screen.dart	Экран настроек приложения

Таблица 2: Структура файлов проекта

4.3 База данных слов

База данных слов реализована в виде класса `WordDatabase`, содержащего статический список объектов `Word`. Каждое слово имеет три атрибута: английское написание, русский перевод и уровень сложности. Всего база содержит 40 слов, распределённых по четырём уровням:

Уровень	Количество слов	Примеры
Начинающий	10	cat, dog, house, book, apple
Средний	10	beautiful, dangerous, knowledge, freedom
Продвинутый	10	extraordinary, consciousness, phenomenon
Профессионал	10	implementation, infrastructure, sustainability

Таблица 3: Распределение слов по уровням

4.4 Реализация игровых режимов

4.4.1 Режим «Сопоставление пар» (Matching)

Режим сопоставления пар отображает два столбца: слева — английские слова, справа — русские переводы в случайном порядке. Пользователь выбирает слово в левом столбце, затем соответствующий перевод в правом. При правильном сопоставлении пара выделяется зелёным цветом и блокируется от дальнейшего выбора.

Алгоритм работы:

1. Из базы данных выбираются 6 случайных слов соответствующего уровня
2. Русские переводы перемешиваются в случайном порядке
3. Пользователь кликает на английское слово (оно подсвечивается)
4. Пользователь кликает на русский перевод
5. Система проверяет соответствие пары
6. При правильном ответе пара блокируется, при неправильном — сбрасывается выбор
7. После составления всех пар подсчитывается результат

4.4.2 Режим «Викторина» (Quiz)

Режим викторины представляет собой классический тест с выбором правильного ответа. Пользователю показывается английское слово и четыре варианта перевода, из которых только один правильный.

Алгоритм работы:

1. Из базы данных выбираются 8 случайных слов соответствующего уровня
2. Для каждого слова генерируются 3 неправильных варианта перевода
3. Варианты ответов перемешиваются в случайном порядке
4. Пользователь выбирает один из вариантов
5. Правильный ответ подсвечивается зелёным, неправильный — красным
6. Через 1 секунду автоматически загружается следующий вопрос
7. После 8 вопросов подсчитывается общий результат

4.4.3 Режим «Перетаскивание» (Drag & Drop)

Режим перетаскивания использует механизм Drag & Drop фреймворка Flutter. Английские слова отображаются как зоны-мишени, а русские переводы — как перетаскиваемые элементы.

Алгоритм работы:

1. Из базы данных выбираются 5 случайных слов соответствующего уровня
2. Английские слова отображаются как контейнеры-мишени
3. Русские переводы отображаются как перетаскиваемые чипы
4. Пользователь перетаскивает русское слово на соответствующий английский контейнер
5. Использованные русские слова скрываются из списка доступных

6. После заполнения всех контейнеров проверяются все ответы
7. Правильные сопоставления выделяются зелёным, неправильные — красным

4.5 Система прогресса и звёзд

Система прогресса основана на механизме звёзд. За каждое выполненное упражнение пользователь получает от 1 до 3 звёзд в зависимости от процента правильных ответов:

Процент правильных ответов	Количество звёзд
90% и выше	3 звезды
70% – 89%	2 звезды
Менее 70%	1 звезда

Таблица 4: Система начисления звёзд

Звёзды сохраняются локально с использованием `SharedPreferences` [7] и загружаются при следующем запуске приложения. Если пользователь улучшает свой результат, новое количество звёзд перезаписывает предыдущее.

4.6 Тёмная и светлая тема

Приложение поддерживает переключение между светлой и тёмной темами. Реализация основана на параметре `themeMode` в `MaterialApp`. Состояние темы хранится в `_MyAppState` и переключается нажатием на иконку в `AppBar`.

Листинг 1: Реализация переключения темы

```
1 void _toggleTheme(bool value) {  
2     setState(() => _isDarkMode = value);  
3 }  
4  
5 // В MaterialApp:  
6 themeMode: _isDarkMode ? ThemeMode.dark : ThemeMode.light,  
7 theme: ThemeData(  
8     brightness: Brightness.light,  
9     primarySwatch: Colors.blue,  
10 ),  
11 darkTheme: ThemeData(  
12     brightness: Brightness.dark,  
13     primarySwatch: Colors.blue,  
14 ),
```

4.7 Голосовое распознавание

Для голосового ввода используется Web Speech API [9], доступный в современных браузерах. Взаимодействие с JavaScript осуществляется через пакет `dart:js`:

Листинг 2: JavaScript-интероп для голосового ввода

```
1 void startSpeechRecognition(String lang, String callbackName) {  
2     try {  
3         js.context.callMethod('startSpeechRecognition', [lang, callbackName]);  
4     } catch (e) {  
5         debugPrint('Ошибка_старта_распознавания:_$e');  
6     }  
7 }
```

JavaScript-функции определены в файле `web/index.html` и обеспечивают захват аудио-потока с микрофона, распознавание речи и передачу результата обратно в Dart-код.

4.8 Серверная часть

Серверная часть приложения реализована на языке Python [10] и представлена файлом `server.py` в корне проекта. Сервер обеспечивает:

- Аутентификацию пользователей
- Хранение профилей и статистики в базе данных PostgreSQL [11]
- Синхронизацию игровых сессий для режима PvP
- Взаимодействие с LLM-моделью для генерации персонализированных упражнений

Взаимодействие между клиентом (Flutter) и сервером (Python) осуществляется через REST API с использованием HTTP-запросов [8].

4.9 Аудиосистема

Приложение включает встроенную аудиосистему, обеспечивающую:

- Фооновую музыку (воспроизводится циклично)
- Звуковые эффекты (щелчки, правильный/неправильный ответ, завершение)
- Регулировку громкости фоновой музыки и звуковых эффектов
- Выбор устройства микрофона для голосового ввода

Аудиофайлы хранятся в директориях `assets/audio/` и `web/` в форматах MP3 и OGG для совместимости с различными браузерами.

5 Тестирование и результаты

5.1 Методология тестирования

Тестирование приложения проводилось на трёх платформах: Android (эмулятор и физическое устройство), Windows (десктоп) и Web (Chrome, Edge). Использовались следующие виды тестирования:

- **Функциональное тестирование** — проверка корректности работы всех функций согласно требованиям
- **Тестирование пользовательского интерфейса** — проверка отображения на разных разрешениях экрана
- **Тестирование совместимости** — проверка работы на разных платформах и браузерах
- **Тестирование производительности** — измерение времени запуска и частоты кадров

5.2 Результаты функционального тестирования

№	Тест	Ожидаемый результат
1	Запуск приложения	Приложение запускается без ошибок
2	Отображение 4 уровней	На главном экране видны 4 кнопки уровней
3	Переход к выбору упражнения	После выбора уровня открывается экран с 3 типами упражнения
4	Режим Matching	Корректное сопоставление 6 пар слов
5	Режим Quiz	8 вопросов с 4 вариантами ответов
6	Режим Drag & Drop	Перетаскивание 5 слов на соответствующие мишени
7	Начисление звёзд	3 звезды при 90%+, 2 при 70-89%, 1 при <70%
8	Сохранение прогресса	Звёзды сохраняются после перезапуска
9	Тёмная тема	Переключение темы через иконку
10	Русский интерфейс	Все надписи на русском языке

Таблица 5: Результаты функционального тестирования

5.3 Тестирование пользовательского интерфейса

Пользовательский интерфейс был протестирован на экранах с различным разрешением: от 360x640 (мобильный телефон) до 1920x1080 (десктоп). Приложение корректно адаптируется под размер экрана благодаря использованию Expanded и Flexible виджетов Flutter.

5.4 Тестирование совместимости

Платформа	Версия	Результат
Android (эмулятор)	13.0	Успешно
Android (физическое устройство)	11.0	Успешно
Windows	10	Успешно
Chrome	148	Успешно
Edge	148	Успешно

Таблица 6: Результаты тестирования совместимости

5.5 Выявленные проблемы и их решение

В процессе тестирования были выявлены следующие проблемы:

1. **Проблема:** Порт 8080 занят предыдущим экземпляром приложения. **Решение:** Использовать команду `taskkill /PID /F` для освобождения порта или указывать другой порт через `--web-port`.
2. **Проблема:** Ошибка компиляции для Windows из-за отсутствия Visual Studio. **Решение:** Запускать веб-версию через `flutter run -d chrome` для тестирования без установки дополнительных инструментов.
3. **Проблема:** Отсутствие поддержки XeLaTeX по умолчанию в Overleaf. **Решение:** Изменить компилятор на XeLaTeX в настройках проекта.
4. **Проблема:** Некорректное отображение кириллицы в консоли. **Решение:** Использовать кодировку UTF-8 и настроить шрифты в LaTeX.

5.6 Производительность

Параметр	Измеренное значение	Норма
Время запуска (Web)	2,8 с	< 3 с
Время запуска (Android)	1,5 с	< 3 с
Частота кадров	60 FPS	60 FPS
Использование памяти (Web)	85 МБ	< 200 МБ

Таблица 7: Показатели производительности

6 Развертывание и использование

6.1 Требования к окружению

Для запуска приложения на локальной машине необходимо следующее программное обеспечение:

- Flutter SDK версии 3.0.0 или выше (рекомендуется стабильная ветка) [5]
- Dart SDK (устанавливается вместе с Flutter) [6]
- Python 3.x (для работы серверной части, опционально) [10]
- Система управления базами данных PostgreSQL (для полной функциональности сервера) [11]
- Веб-браузер Chrome версии 90+ или Edge версии 90+

6.2 Клонирование репозитория

Исходный код проекта размещён в публичном репозитории GitHub. Для получения копии необходимо выполнить команду:

Листинг 3: Клонирование репозитория

```
1 git clone https://github.com/EfecTr0/Duolingo
2 cd Duolingo/game_app
```

6.3 Установка зависимостей Flutter

После перехода в директорию `game_app` необходимо установить все зависимости, указанные в файле `pubspec.yaml`:

Листинг 4: Установка зависимостей

```
1 flutter pub get
```

6.4 Запуск приложения в веб-режиме

Наиболее простой способ запуска — использование веб-сервера Flutter. Выполните одну из следующих команд:

- `flutter run -d chrome` — автоматически откроет приложение в браузере Chrome
- `flutter run -d web-server --web-port=8080` — запустит веб-сервер на порту 8080, после чего приложение будет доступно по адресу `http://localhost:8080/`

При возникновении ошибки «Failed to bind web development server» (порт занят) следует либо освободить порт командой `taskkill /PID <номер> /F`, либо указать другой порт, например `--web-port=5555`.

6.5 Запуск серверной части

Для полноценной работы функций аутентификации, PvP и AI-генерации необходимо запустить серверную часть на Python. Убедитесь, что у вас установлен Python 3.x и необходимые библиотеки, затем выполните:

Листинг 5: Запуск сервера

```
1 cd ..  
2 python server.py
```

Сервер будет ожидать подключения от клиентского приложения Flutter. При необходимости настройте параметры подключения к базе данных PostgreSQL в конфигурационном файле сервера.

6.6 Использование приложения

После успешного запуска пользователь попадает на экран аутентификации. Далее основные шаги:

1. Войти в систему, используя учётные данные.
2. На главном экране выбрать один из четырёх уровней сложности.
3. На экране выбора упражнения выбрать тип тренировки (Matching, Quiz, Drag & Drop).
4. Выполнить упражнение, следуя инструкциям на экране.
5. Просмотреть результат (количество звёзд) и вернуться к выбору уровня.
6. Для соревновательного режима перейти в раздел «Друзья», пригласить друга и начать PvP-батл.

Настройки темы и громкости доступны через соответствующий пункт меню в любой момент.

6.7 Возможные неполадки и их устранение

- **Ошибка компиляции для Windows:** требуется установить Visual Studio с компонентами «Desktop development with C++», либо использовать веб-версию.
- **Не работает голосовой ввод:** Web Speech API поддерживается только в Chrome и Edge; предоставьте браузеру доступ к микрофону.
- **Сервер не отвечает:** проверьте, что `server.py` запущен и база данных PostgreSQL активна; проверьте настройки брандмауэра.

7 Заключение

7.1 Итоги работы

В рамках данного проекта было разработано кроссплатформенное интерактивное приложение для изучения английского языка на базе фреймворка Flutter [5]. Приложение ориентировано на русскоговорящих пользователей и предоставляет три типа упражнений на четырёх уровнях сложности.

В ходе выполнения проекта были достигнуты следующие результаты:

1. Проведён анализ рынка образовательных приложений и выявлены основные конкуренты (Duolingo, Babbel, Lingualeo)
2. Изучены теоретические основы Flutter, Dart, SharedPreferences и принципов геймификации
3. Спроектирована модульная архитектура приложения
4. Реализован пользовательский интерфейс на русском языке с поддержкой светлой и тёмной тем
5. Разработана база данных из 40 слов, распределённых по четырём уровням сложности
6. Реализованы три интерактивных упражнения: сопоставление пар, викторина и перетаскивание
7. Внедрена система сохранения прогресса на основе звёзд
8. Проведено тестирование на трёх платформах (Android, Windows, Web)

7.2 Практическая значимость

Разработанное приложение может использоваться:

- Студентами для самостоятельного изучения английской лексики
- Преподавателями в качестве дополнительного инструмента обучения
- Разработчиками как основа для создания более сложных образовательных приложений на Flutter

7.3 Возможные улучшения

В будущем приложение может быть расширено следующими возможностями:

1. Добавление звукового сопровождения для произношения слов (озвучивание)
2. Внедрение таймера для каждого упражнения с бонусными очками
3. Разработка детальной статистики с графиками прогресса
4. Добавление поддержки других языковых пар (французский, немецкий, испанский)
5. Режим повторения слов, в которых пользователь допустил ошибки
6. Система достижений и ежедневных заданий
7. Интеграция с LLM-моделями для генерации персонализированных упражнений
8. Полноценная серверная часть с многопользовательским режимом

7.4 Заключительные выводы

Проект продемонстрировал эффективность использования фреймворка Flutter для быстрой разработки кроссплатформенных образовательных приложений. Полученное приложение полностью соответствует поставленным требованиям и может быть использовано как в учебных целях, так и как основа для дальнейшего коммерческого развития.

Список литературы

- [1] Ethnologue. Languages of the World, 28th Edition. — SIL International, 2025.
- [2] Statista. Language Learning App Market Report 2025. [Электронный ресурс]. URL: <https://www.statista.com/> (дата обращения: 10.06.2026).
- [3] Duolingo Official Website. [Электронный ресурс]. URL: <https://www.duolingo.com/> (дата обращения: 01.06.2026).
- [4] Babbel Official Website. [Электронный ресурс]. URL: <https://www.babbel.com/> (дата обращения: 01.06.2026).
- [5] Flutter Documentation. [Электронный ресурс]. URL: <https://docs.flutter.dev/> (дата обращения: 01.06.2026).
- [6] Dart Documentation. [Электронный ресурс]. URL: <https://dart.dev/guides> (дата обращения: 01.06.2026).
- [7] SharedPreferences Plugin. [Электронный ресурс]. URL: https://pub.dev/packages/shared_preferences (дата обращения: 05.06.2026).
- [8] HTTP Package for Dart. [Электронный ресурс]. URL: <https://pub.dev/packages/http> (дата обращения: 05.06.2026).
- [9] MDN Web Docs. Web Speech API. [Электронный ресурс]. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API (дата обращения: 10.06.2026).
- [10] Python Software Foundation. Python 3.x Documentation. [Электронный ресурс]. URL: <https://docs.python.org/3/> (дата обращения: 01.06.2026).
- [11] PostgreSQL Documentation. [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 01.06.2026).
- [12] Deterding S., Dixon D., Khaled R., Nacke L. From Game Design Elements to Gamefulness: Defining «Gamification» // MindTrek '11. — ACM, 2011. — P. 9–15.
- [13] Hamari J., Koivisto J., Sarsa H. Does Gamification Work? — A Literature Review of Empirical Studies on Gamification // HICSS '14. — IEEE, 2014. — P. 3025–3034.
- [14] Google. Material Design Guidelines. [Электронный ресурс]. URL: <https://material.io/design> (дата обращения: 01.06.2026).
- [15] Flutter Community. Best Practices for Flutter App Development. [Электронный ресурс]. URL: <https://flutter.dev/docs/development> (дата обращения: 05.06.2026).